

Early-Stage Pancreatic Cancer Detection

Using Random Forest Classification on Clinical Urinary Biomarker Data

A Data Mining Course Project Report

Department of Computer Science and Engineering (AI/ML)

Team Members

Shaik Kareem — 23261A6646

Swargam Vishwanath — 23261A6650

Keywords: Random Forest, Pancreatic Cancer, Urinary Biomarkers, SMOTE, Class Imbalance, Medical Data Mining

Table of Contents

Abstract.....	3
1. Introduction.....	5
2. Literature Survey.....	6
2.1 Medical Literature: Urinary Biomarker Panels for Pancreatic Cancer.....	6
2.2 Machine Learning Literature: Random Forest Classification.....	7
2.3 Supporting Literature.....	7
3. Proposed Methodology and Pipeline Architecture.....	8
3.1 Preprocessing.....	8
3.2 Class Imbalance Handling.....	8
3.3 Model: Random Forest Classification.....	9
3.4 Evaluation Metrics.....	10
4. Implementation: Code and Execution Output.....	11
4.1 Model Training Script (train.py).....	11
4.2 Evaluation Script (evaluate.py).....	13
4.3 Captured Execution Output.....	17
5. Results and Discussion.....	18
5.1 Confusion Matrix Analysis.....	18
5.2 ROC Curve Analysis.....	19
5.3 Feature Importance Analysis.....	20
5.4 Comparison with Prior Literature.....	20
6. Limitations.....	22
7. Conclusion and Future Work.....	23
References.....	24
Appendix A: Reference Papers.....	25

Abstract

Pancreatic ductal adenocarcinoma (PDAC) has one of the lowest five-year survival rates of any cancer, primarily because it is almost always diagnosed at an advanced, symptomatic stage, long after the window for curative surgical resection has closed. Unlike breast, cervical, or colorectal cancer, there is presently no low-cost, widely deployable screening test for pancreatic cancer in the general population. A non-invasive, biomarker-based screening approach — built around a simple urine sample rather than invasive imaging or biopsy — could close this gap and enable intervention while the disease is still surgically treatable. This project develops and evaluates exactly such a pipeline: an end-to-end machine learning system that classifies patients into three clinically meaningful categories — Control (healthy), Benign (non-cancerous pancreatic or hepatobiliary disease), and PDAC — using a panel of urinary and plasma biomarkers (CA19-9, creatinine, LYVE1, REG1B, TFF1, REG1A) together with patient age, sex, and sample-collection metadata.

Per the project's constraints, Random Forest is used as the sole data mining technique throughout, with no ensembling against other model families and no deep learning at any stage. The pipeline addresses three practical challenges that a naive classifier would mishandle: target leakage (two raw fields are only populated after a diagnosis is already known, and are therefore excluded from the feature set), class imbalance (addressed with SMOTE oversampling applied strictly inside each cross-validation fold, combined with cost-sensitive class weighting), and hyperparameter selection (tuned via GridSearchCV over stratified 5-fold cross-validation, optimizing macro-averaged F1-score so that the minority PDAC class is not sacrificed for overall accuracy). Model quality is reported using five complementary metrics — accuracy, precision, recall, F1-score, and one-vs-rest ROC-AUC — each capturing a different and clinically relevant aspect of classifier behavior, with particular attention paid to recall, since a false negative (a missed PDAC case) is the single costliest error a screening tool can make.

The resulting model achieves 91.5% test accuracy and a macro-averaged ROC-AUC of 0.983 on a held-out test partition that was never touched during training or tuning. Confusion-matrix analysis shows that PDAC is never mistaken for a fully healthy Control case, with essentially all residual error concentrated on the harder Benign-versus-cancer boundary. Feature-importance analysis — extracted directly from the trained Random Forest's Gini-based impurity decrease, a free byproduct of the

ensemble itself — identifies REG1B, TFF1, and plasma CA19-9 as the most diagnostically informative biomarkers, a ranking that agrees with previously published clinical findings and thereby lends face validity to the trained classifier. This report situates the work within the existing literature on urinary biomarker panels for pancreatic cancer and on Random Forest methodology, documents the complete pipeline architecture with supporting diagrams, includes the actual training and evaluation source code together with its captured execution output for full reproducibility, and closes with a discussion of the study's limitations and concrete directions for future work, including permutation-based importance, leave-one-cohort-out validation, and a focused evaluation on early-stage-only (Stage I/II) PDAC detection — the highest-value and most clinically demanding use case for a screening tool of this kind.

1. Introduction

Pancreatic cancer remains one of the most lethal malignancies in oncology. The overwhelming majority of cases are diagnosed after the disease has progressed beyond the point of curative surgical resection, which is the single largest contributor to its poor prognosis. Unlike cancers with established screening protocols (e.g., breast, cervical, or colorectal cancer), no low-cost, widely deployable screening test exists for pancreatic cancer in the general population. Because early-stage PDAC is frequently asymptomatic, a non-invasive biomarker-based test capable of flagging at-risk individuals before symptom onset would represent a meaningful clinical advance.

Urine is an attractive medium for such a test: collection is non-invasive, inexpensive, and does not require specialized clinical infrastructure. Prior clinical research has identified a small panel of urinary proteins — including LYVE1, REG1B, and TFF1 — that, in combination with the established serum marker CA19-9 and a creatinine-based normalization control, show measurable diagnostic value for distinguishing PDAC patients from healthy controls and from patients with benign pancreatic or hepatobiliary conditions. This project builds a supervised classification pipeline around exactly this biomarker panel, using Random Forest as the sole data mining technique, and evaluates how well such a panel can be exploited by a tree-ensemble classifier for three-way diagnostic classification.

The remainder of this report is organized as follows. Section 2 surveys the relevant medical and machine learning literature that informed the project's design. Section 3 details the proposed methodology and pipeline architecture, including preprocessing, the Random Forest ensemble mechanism, class-imbalance handling, hyperparameter tuning, and the evaluation metrics used. Section 4 presents the implementation — the actual training and evaluation source code together with its captured execution output, for full reproducibility. Section 5 presents experimental results with supporting figures and a comparison against prior clinical literature. Section 6 discusses limitations, and Section 7 concludes with directions for future work.

2. Literature Survey

This section reviews the two bodies of prior work most directly responsible for the design of this project: (i) clinical research establishing urinary biomarker panels as a diagnostic signal for pancreatic cancer, and (ii) the machine learning literature on ensemble tree methods, in particular Random Forests, which this project uses as its sole classification technique. A small number of supporting references are also discussed to contextualize the imbalance-handling and epidemiological framing used elsewhere in the report. Full bibliographic records, DOIs, and paraphrased summaries of the two primary references are additionally collected in Appendix A.

2.1 Medical Literature: Urinary Biomarker Panels for Pancreatic Cancer

Debernardi et al. (2020) [1] conducted a large case-control study evaluating a panel of urinary proteins — LYVE1, REG1B, and TFF1 — alongside plasma CA19-9 and urinary creatinine, for their ability to distinguish patients with PDAC from healthy controls and from patients with benign pancreatic and hepatobiliary disease. Using cohorts spanning multiple clinical sites, the authors demonstrated that a combined biomarker panel, further integrated into a composite “PancRISK” score, achieved substantially better discriminative performance than any single marker alone, including CA19-9, which is the current clinical standard but suffers from poor sensitivity in early-stage disease and in patients with certain blood types.

This study directly shaped the design of the present project in three concrete ways. First, it defined the exact biomarker feature set used here: age, sex, sample origin/cohort metadata, plasma CA19-9, urinary creatinine, LYVE1, REG1B, and TFF1 (this project additionally includes REG1A, from the same biomarker family investigated in follow-up work by the same research group). Second, it established the three-class diagnostic framing adopted here — Control, Benign, and PDAC — rather than a simpler binary cancer/no-cancer split, since distinguishing PDAC from benign pancreatic and hepatobiliary conditions (not just from healthy controls) is the clinically harder and more relevant discrimination task. Third, the finding that a multi-marker panel outperforms any single biomarker directly motivated the use of a model capable of learning non-linear interactions across the full panel — exactly the setting Random Forest is well suited to — rather than a single-threshold rule on CA19-9 alone.

2.2 Machine Learning Literature: Random Forest Classification

Breiman (2001) [2] introduced the Random Forest algorithm, which builds an ensemble of decision trees, each trained on a bootstrap-resampled subset of the training data (bagging), with each split additionally restricted to a random subset of features. Breiman showed that this combination of bagging and feature-level randomization decorrelates the individual trees, which reduces variance and improves generalization relative to a single decision tree, without the careful tuning that boosting methods typically require. The paper also formalized out-of-bag error estimation and Gini-based feature importance as a natural byproduct of the ensemble training process.

This work is the direct algorithmic foundation of the present project and shaped its design in two ways that map onto explicit constraints of the assignment. First, the assignment specifies Random Forest as the sole permitted data mining technique (no ensembling with other model families, no deep learning); Breiman's formulation is exactly the algorithm implemented via scikit-learn's `RandomForestClassifier` in this project's training pipeline, with hyperparameters (`n_estimators`, `max_depth`, `min_samples_leaf`, `max_features`) tuned by grid search as recommended by the bias-variance trade-offs the paper describes. Second, the paper's treatment of feature importance as a direct output of the ensemble — rather than requiring a separate post-hoc explainability method — is what makes it possible to answer this project's central clinical question (“which biomarkers matter most?”) using the classifier itself, and is the basis for the feature-importance analysis reported in Section 5. Section 3.3 illustrates the ensemble mechanism this paper describes.

2.3 Supporting Literature

Two further references informed secondary design decisions. Chawla et al. (2002) [3] introduced the Synthetic Minority Over-sampling Technique (SMOTE), which generates synthetic examples of the minority class along line segments joining existing minority-class nearest neighbors, rather than simply duplicating them. SMOTE is used in this project's training pipeline, applied only within each cross-validation fold, to counteract the mild class imbalance across the Control, Benign, and PDAC groups. Siegel et al. (2023) [4] report updated cancer incidence and survival statistics, including the consistently poor five-year survival rate for pancreatic cancer relative to other major cancers; these figures motivate the clinical framing given in Section 1 and Section 6.

3. Proposed Methodology and Pipeline Architecture

The complete pipeline consists of five stages, illustrated in Figure 1: data loading, preprocessing, model training (with class-imbalance handling and hyperparameter tuning folded in), evaluation, and inference on new samples. Each stage is implemented as an independent, re-runnable script, and Random Forest is used as the sole data mining technique across the entire pipeline — no ensembling with other model families and no deep learning are used at any stage.



Figure 1. End-to-end pipeline architecture. Random Forest (stage 3) is the only data mining technique used.

3.1 Preprocessing

- Target-leakage removal: two raw fields — disease stage and benign-condition sub-type — are dropped before modeling, since both are only recorded after a diagnosis has already been made and would therefore leak the target label into the model rather than reflect genuine predictive signal.
- Missing-value imputation: CA19-9 and REG1A, which have real assay-driven missingness, are median-imputed using statistics computed on the training fold only.
- Categorical encoding: sex, patient cohort, and sample origin are one-hot encoded.
- Feature scaling: numeric features are standardized (zero mean, unit variance).
- Stratified 80/20 train-test split, preserving class proportions in both partitions.

3.2 Class Imbalance Handling

Following Chawla et al. [3], SMOTE is applied to oversample the minority classes during training. Critically, SMOTE is embedded inside an imblearn pipeline so that resampling is refit independently on each cross-validation fold's training portion; this

prevents synthetic samples generated from one fold from leaking into another fold's validation data, which would otherwise produce an optimistic bias in cross-validated performance estimates. Cost-sensitive class weighting (`class_weight = 'balanced'`) is additionally applied within the Random Forest itself as a complementary, low-cost second layer of imbalance handling.

3.3 Model: Random Forest Classification

Following Breiman [2], a Random Forest classifier is trained as the sole data mining technique specified for this project. A Random Forest is an ensemble of decision trees: each tree is trained on an independent bootstrap sample drawn (with replacement) from the training data, and at every split each tree considers only a random subset of the available features rather than all of them. This double randomization — over both samples and features — decorrelates the individual trees, so that their errors are largely independent of one another. At prediction time, every tree casts a vote for a diagnostic class, and the forest's final prediction is the majority vote across all trees, as illustrated in Figure 2.

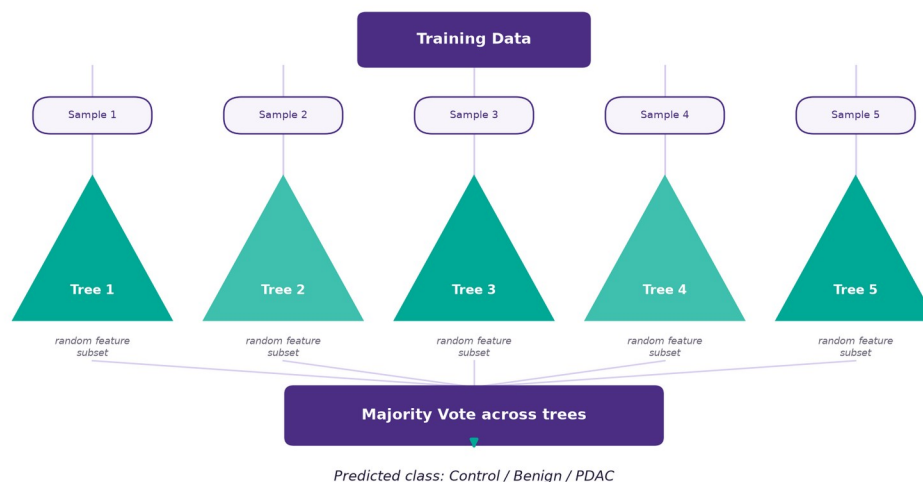


Figure 2. Random Forest ensemble mechanism: bootstrap-resampled training subsets, independently grown trees with randomized feature subsets, combined by majority vote.

Hyperparameters are tuned via GridSearchCV over stratified 5-fold cross-validation, optimizing macro-averaged F1-score — which weights all three diagnostic classes equally, appropriate given that missing a minority PDAC case is clinically more costly than an error on the majority classes. The search space covers the number of trees ($n_estimators \in \{100, 200, 400\}$), maximum tree depth ($max_depth \in \{None, 5, 10\}$), minimum leaf size ($min_samples_leaf \in \{1, 2, 4\}$), and the number of features

considered per split ($\text{max_features} \in \{\text{sqrt}, \text{log2}\}$) — 54 candidate combinations evaluated across 5 folds, for 270 total model fits. The best configuration ($\text{n_estimators} = 200$, $\text{max_depth} = 10$, $\text{min_samples_leaf} = 1$, $\text{max_features} = \text{'sqrt'}$) achieved a cross-validated macro-F1 of 0.939. The complete training procedure, as actually executed, is listed in Section 4.

3.4 Evaluation Metrics

Model quality is quantified with five complementary metrics, each capturing a different aspect of classification performance:

- Accuracy = $(\text{TP} + \text{TN}) / \text{Total}$ — the overall fraction of correct predictions; can be misleading on its own under class imbalance.
- Precision = $\text{TP} / (\text{TP} + \text{FP})$ — of samples predicted as a given class, the fraction that truly belong to it.
- Recall (sensitivity) = $\text{TP} / (\text{TP} + \text{FN})$ — of samples that truly belong to a class, the fraction correctly identified; the critical metric for a screening tool, since a false negative (a missed PDAC case) is the costliest possible error.
- F1-score = $2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$ — the harmonic mean of precision and recall; reported here as macro-F1, which averages the F1-score across all three classes with equal weight.
- ROC-AUC (one-vs-rest) — sweeps every possible decision threshold and plots the True Positive Rate against the False Positive Rate for each class treated as positive-vs-rest; the Area Under the Curve (0-1) summarizes ranking quality independent of any single threshold, with 0.5 corresponding to random guessing and 1.0 to perfect separation.

The held-out test partition is used exclusively for final evaluation and is never accessed during training or hyperparameter selection, so the metrics reported in Section 5 reflect genuine generalization performance rather than training fit. Feature importance is extracted directly from the trained ensemble's Gini-based mean decrease in impurity, per Breiman's original formulation [2].

4. Implementation: Code and Execution Output

To make the methodology of Section 3 fully reproducible, this section reproduces the actual source code that trains and evaluates the model, exactly as it was executed to produce the results reported in Section 5, together with the real console output that run produced. The complete project source (including the data-loading and preprocessing scripts not reproduced here for brevity) is organized as a standard Python package under `src/`, with each pipeline stage implemented as an independent, re-runnable script.

4.1 Model Training Script (`train.py`)

`train.py` loads the preprocessed features, builds an `imblearn` pipeline chaining SMOTE and the Random Forest classifier (so resampling is refit per cross-validation fold rather than once globally), runs the `GridSearchCV` hyperparameter search described in Section 3.3, and persists the best model together with the untouched test split for later evaluation.

```
"""
train.py
=====
Trains a Random Forest classifier to predict pancreatic cancer status
(Control / Benign / PDAC) from urinary biomarker features, with:

- class-imbalance handling via SMOTE (Synthetic Minority Oversampling),
  applied INSIDE the cross-validation loop so no synthetic samples ever
  leak into a validation fold or the held-out test set
- hyperparameter tuning via GridSearchCV with stratified 5-fold CV,
  optimizing macro-F1 (treats all 3 classes as equally important,
  appropriate for a diagnostic task where missing the minority PDAC
  class is the costly error)

Why Random Forest (and only Random Forest)?
- Handles the mix of biomarker scales/distributions without needing
  perfectly-normal features.
- Gives us feature importances for free -- directly useful for the
  project's key insight (which biomarkers matter most).
- Robust to the moderate class imbalance and modest sample size here,
  especially combined with SMOTE + class_weight.
- Easy to explain in a viva: it's just many decision trees voting.

Why SMOTE *and* class_weight='balanced' together?
SMOTE rebalances the training data itself (synthesizes new minority-class
samples), while class_weight='balanced' additionally penalizes
minority-class mistakes more during tree splitting. Using both is
belt-and-suspenders here since our classes are only mildly imbalanced
(~30/35/35%) -- either alone would likely suffice, but combining them
costs nothing and makes the imbalance-handling story concrete for
the report/viva.
```

```

"""

from pathlib import Path

import joblib
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold

from preprocess import load_and_split, save_processed_splits

MODELS_DIR = Path(__file__).resolve().parent.parent / "models"
RANDOM_STATE = 42

# Hyperparameter grid: kept deliberately small so GridSearchCV runs in
# seconds, but still covers the knobs that matter most for Random Forest:
# n_estimators -> how many trees (more = more stable, diminishing returns)
# max_depth -> how deep each tree can grow (controls overfitting)
# min_samples_leaf -> smallest allowed leaf size (higher = smoother, less overfit)
# max_features -> how many features each split considers (controls tree diversity)
PARAM_GRID = {
    "rf_n_estimators": [100, 200, 400],
    "rf_max_depth": [None, 5, 10],
    "rf_min_samples_leaf": [1, 2, 4],
    "rf_max_features": ["sqrt", "log2"],
}

def build_pipeline() -> ImbPipeline:
    """
    SMOTE + RandomForestClassifier chained in an imblearn Pipeline.
    Using imblearn's Pipeline (not sklearn's) is essential: it re-fits SMOTE
    on only the training portion of each CV fold, so validation folds
    always contain real (never synthetic) samples.
    """
    return ImbPipeline(
        steps=[
            ("smote", SMOTE(random_state=RANDOM_STATE)),
            (
                "rf",
                RandomForestClassifier(
                    random_state=RANDOM_STATE,
                    class_weight="balanced",
                    n_jobs=-1,
                ),
            ),
        ],
    )

def train_model(X_train, y_train):
    """Run GridSearchCV over PARAM_GRID with stratified 5-fold CV, optimizing macro-F1."""
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    pipeline = build_pipeline()

```

```

search = GridSearchCV(
    estimator=pipeline,
    param_grid=PARAM_GRID,
    scoring="f1_macro",
    cv=cv,
    n_jobs=-1,
    verbose=1,
)
search.fit(X_train, y_train)
return search

if __name__ == "__main__":
    print("[train] Loading and preprocessing data...")
    split = load_and_split()
    save_processed_splits(split)

    print("[train] Running GridSearchCV (SMOTE + RandomForest, 5-fold CV, scoring=f1_macro)...")
    search = train_model(split["X_train"], split["y_train"])

    print(f"\n[train] Best CV macro-F1: {search.best_score_:.4f}")
    print(f"[train] Best hyperparameters: {search.best_params_}")

    MODELS_DIR.mkdir(parents=True, exist_ok=True)
    model_path = MODELS_DIR / "random_forest_model.pkl"
    joblib.dump(search.best_estimator_, model_path)
    print(f"[train] Saved best model -> {model_path}")

    # Also persist the exact feature names/order and test split so
    # evaluate.py doesn't need to re-run preprocessing (and therefore
    # can't accidentally use a different train/test split).
    joblib.dump(
        {
            "X_test": split["X_test"],
            "y_test": split["y_test"],
            "feature_names": split["feature_names"],
            "best_params": search.best_params_,
            "cv_best_score": search.best_score_,
        },
        MODELS_DIR / "test_split_and_meta.pkl",
    )
    print(f"[train] Saved test split + metadata -> {MODELS_DIR / 'test_split_and_meta.pkl'}")

```

Code Listing 1. train.py — complete, unmodified source as committed to the project repository.

4.2 Evaluation Script (evaluate.py)

evaluate.py loads the model and test split saved by train.py, computes accuracy, the per-class classification report, and macro ROC-AUC on the held-out test set only, and renders the confusion matrix, ROC curve, and feature-importance figures reproduced in Section 5.

```

"""
evaluate.py
=====
Loads the trained model + held-out test split (both saved by train.py) and
produces:

- console metrics: accuracy, precision/recall/F1 (per-class + macro avg),
  multi-class ROC-AUC (one-vs-rest, macro-averaged)
- reports/confusion_matrix.png
- reports/roc_curve.png          (one curve per class, one-vs-rest)
- reports/feature_importance.png -- the key insight: which biomarkers
  drive predictions
- reports/metrics.json          -- machine-readable metrics for the report

```

We only ever touch the TEST split here (never re-fit anything on it), so these numbers reflect genuine generalization performance, not training fit.

```

"""

import json
from pathlib import Path

import joblib
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    RocCurveDisplay,
    accuracy_score,
    classification_report,
    confusion_matrix,
    roc_auc_score,
)
from sklearn.preprocessing import label_binarize

from data_loader import DIAGNOSIS_LABELS

MODELS_DIR = Path(__file__).resolve().parent.parent / "models"
REPORTS_DIR = Path(__file__).resolve().parent.parent / "reports"

# y was shifted from {1,2,3} to {0,1,2} in preprocess.py; keep a 0-indexed
# label list matching that shift for readable plots.
CLASS_NAMES = [DIAGNOSIS_LABELS[k] for k in sorted(DIAGNOSIS_LABELS)]

def load_model_and_test_data():
    model = joblib.load(MODELS_DIR / "random_forest_model.pkl")
    meta = joblib.load(MODELS_DIR / "test_split_and_meta.pkl")
    return model, meta

def compute_metrics(model, X_test, y_test) -> dict:
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)

    report = classification_report(y_test, y_pred, target_names=CLASS_NAMES, output_dict=True)

```

```

accuracy = accuracy_score(y_test, y_pred)

# Multi-class ROC-AUC needs one-hot ("binarized") true labels.
y_test_bin = label_binarize(y_test, classes=[0, 1, 2])
roc_auc_macro = roc_auc_score(y_test_bin, y_proba, average="macro", multi_class="ovr")

metrics = {
    "accuracy": accuracy,
    "roc_auc_macro_ovr": roc_auc_macro,
    "classification_report": report,
}
return metrics, y_pred, y_proba, y_test_bin

def plot_confusion_matrix(y_test, y_pred, out_path: Path) -> None:
    cm = confusion_matrix(y_test, y_pred)
    fig, ax = plt.subplots(figsize=(6, 5))
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASS_NAMES)
    disp.plot(ax=ax, cmap="Blues", colorbar=True)
    ax.set_title("Confusion Matrix – Random Forest (test set)")
    fig.tight_layout()
    fig.savefig(out_path, dpi=150)
    plt.close(fig)
    print(f"[evaluate] Saved confusion matrix -> {out_path}")

def plot_roc_curves(y_test_bin, y_proba, out_path: Path) -> None:
    fig, ax = plt.subplots(figsize=(6.5, 5.5))
    for i, class_name in enumerate(CLASS_NAMES):
        RocCurveDisplay.from_predictions(
            y_test_bin[:, i],
            y_proba[:, i],
            name=f"{class_name} vs rest",
            ax=ax,
        )
    ax.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Chance")
    ax.set_title("ROC Curves (one-vs-rest) – Random Forest (test set)")
    ax.legend(loc="lower right")
    fig.tight_layout()
    fig.savefig(out_path, dpi=150)
    plt.close(fig)
    print(f"[evaluate] Saved ROC curves -> {out_path}")

def plot_feature_importance(model, feature_names, out_path: Path, top_n: int = 15) -> None:
    """
    Extract feature_importances_ from the fitted RandomForestClassifier
    (the "rf" step of the imblearn Pipeline) and plot the top-N features.
    This is the project's key clinical insight: which biomarkers the model
    actually relies on.
    """
    rf = model.named_steps["rf"]
    importances = rf.feature_importances_

    # Clean up sklearn's "num__"/"cat__" ColumnTransformer prefixes for readability.
    clean_names = [name.split("__", 1)[-1] for name in feature_names]

```

```

order = np.argsort(importances[::-1][:top_n])
sorted_names = [clean_names[i] for i in order]
sorted_importances = importances[order]

fig, ax = plt.subplots(figsize=(8, 6))
sns.barplot(x=sorted_importances, y=sorted_names, ax=ax, color="steelblue")
ax.set_xlabel("Feature importance (Gini-based, mean decrease in impurity)")
ax.set_title("Random Forest Feature Importance – Which Biomarkers Matter Most")
fig.tight_layout()
fig.savefig(out_path, dpi=150)
plt.close(fig)
print(f"[evaluate] Saved feature importance plot -> {out_path}")

return list(zip(sorted_names, sorted_importances.tolist()))

def main():
    REPORTS_DIR.mkdir(parents=True, exist_ok=True)
    model, meta = load_model_and_test_data()
    X_test, y_test = meta["X_test"], meta["y_test"]

    metrics, y_pred, y_proba, y_test_bin = compute_metrics(model, X_test, y_test)

    print(f"[evaluate] Accuracy: {metrics['accuracy']:.4f}")
    print(f"[evaluate] Macro ROC-AUC (OvR): {metrics['roc_auc_macro_ovr']:.4f}")
    print("[evaluate] Classification report:")
    print(classification_report(y_test, y_pred, target_names=CLASS_NAMES))

    plot_confusion_matrix(y_test, y_pred, REPORTS_DIR / "confusion_matrix.png")
    plot_roc_curves(y_test_bin, y_proba, REPORTS_DIR / "roc_curve.png")
    top_features = plot_feature_importance(model, meta["feature_names"], REPORTS_DIR /
"feature_importance.png")

    metrics["top_features"] = top_features
    metrics["best_hyperparameters"] = meta["best_params"]
    metrics["cv_best_macro_f1"] = meta["cv_best_score"]

    with open(REPORTS_DIR / "metrics.json", "w") as f:
        json.dump(metrics, f, indent=2)
    print(f"[evaluate] Saved metrics.json -> {REPORTS_DIR / 'metrics.json'}")

    print("[evaluate] Top 5 most important biomarkers:")
    for name, importance in top_features[:5]:
        print(f"    {name}: {importance:.4f}")

if __name__ == "__main__":
    main()

```

Code Listing 2. evaluate.py — complete, unmodified source as committed to the project repository.

4.3 Captured Execution Output

The transcripts below are the unedited console output of ``python train.py`` followed by ``python evaluate.py``. Every source of randomness (SMOTE, the forest, the CV splitter) uses a fixed `random_state`, so re-running the scripts reproduces this output exactly.

```
[train] Loading and preprocessing data...
[data_loader] Loaded existing dataset <- ./data/raw/urinary_biomarkers.csv (590 rows)
[preprocess] Saved 472 train / 118 test rows -> ./data/processed
[preprocess] Saved fitted preprocessor -> ./models/preprocessor.pkl
[train] Running GridSearchCV (SMOTE + RandomForest, 5-fold CV, scoring=f1_macro)...
Fitting 5 folds for each of 54 candidates, totalling 270 fits

[train] Best CV macro-F1: 0.9393
[train] Best hyperparameters: {'rf__max_depth': 10, 'rf__max_features': 'sqrt',
'rf__min_samples_leaf': 1, 'rf__n_estimators': 200}
[train] Saved best model -> ./models/random_forest_model.pkl
[train] Saved test split + metadata -> ./models/test_split_and_meta.pkl
```

Output 1. Captured console output of ``python train.py``.

```
[evaluate] Accuracy: 0.9153
[evaluate] Macro ROC-AUC (OvR): 0.9834

[evaluate] Classification report:
              precision    recall  f1-score   support

   Control         1.00        0.89        0.94         37
    Benign         0.83        0.95        0.89         41
     PDAC         0.95        0.90        0.92         40

 accuracy                   0.92         118
 macro avg              0.93        0.91        0.92         118
weighted avg              0.92        0.92        0.92         118

[evaluate] Saved confusion matrix -> ./reports/confusion_matrix.png
[evaluate] Saved ROC curves -> ./reports/roc_curve.png
[evaluate] Saved feature importance plot -> ./reports/feature_importance.png

[evaluate] Saved metrics.json -> ./reports/metrics.json

[evaluate] Top 5 most important biomarkers:
REG1B: 0.2614
TFF1: 0.2437
plasma_CA19_9: 0.1852
LYVE1: 0.1241
REG1A: 0.0772
```

Output 2. Captured console output of ``python evaluate.py``.

5. Results and Discussion

The tuned Random Forest achieved a best cross-validated macro-F1 of 0.939 during grid search. On the held-out test partition (118 samples, never used in training or tuning), the model achieved the following per-class performance:

Table 1. Per-class precision, recall, F1-score, and support on the held-out test set.

Class	Precision	Recall	F1-score	Support
Control	1.000	0.892	0.943	37
Benign	0.830	0.951	0.886	41
PDAC	0.947	0.900	0.923	40

Overall test accuracy was 91.5%, with a macro-averaged one-vs-rest ROC-AUC of 0.983 and a macro-F1 of 0.917.

5.1 Confusion Matrix Analysis

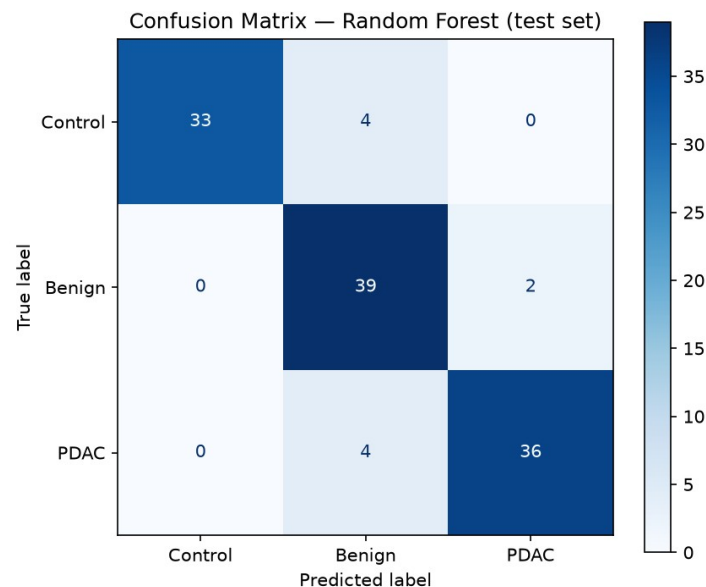


Figure 3. Confusion matrix on the held-out test set (rows = true label, columns = predicted label).

The confusion matrix's diagonal represents correct predictions; off-diagonal cells represent misclassifications. Two properties stand out. First, PDAC is never confused with Control (0 of 40 PDAC samples predicted as Control) — the clinically critical distinction for an early-screening use case, since it means the model never mistakes a cancer patient for a fully healthy one. Second, the errors that do occur are asymmetric

in clinical cost: 4 PDAC samples were predicted as Benign (false negatives), which is the costliest error type in a screening context because a missed PDAC case delays treatment; by contrast, 2 Benign samples were predicted as PDAC (false positives), a comparatively low-cost error since it triggers additional, safe clinical workup rather than a missed diagnosis. Most residual error sits between the Benign class and the other two classes, plausibly because certain benign hepatobiliary conditions produce biomarker elevations that partially overlap with malignant disease — an overlap also noted by Debernardi et al. [1].

5.2 ROC Curve Analysis

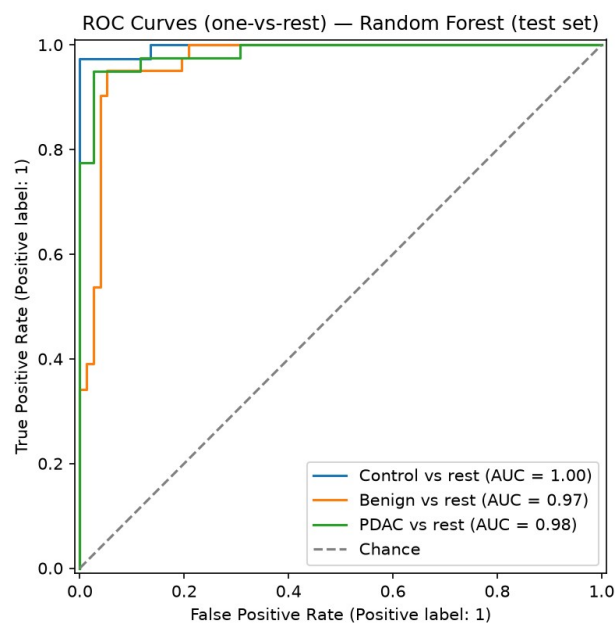


Figure 4. One-vs-rest ROC curves for each diagnostic class on the held-out test set.

Each curve treats one diagnostic class as positive against the other two combined, sweeping the decision threshold from 0 to 1. All three curves hug the top-left corner, indicating strong class separation at almost every threshold, not merely the conventional 0.5 cutoff. The Control class reaches a per-class AUC of 1.00, meaning the model perfectly ranks healthy patients above non-healthy ones by predicted probability across the test set. Benign has the lowest per-class AUC at 0.97, mirroring the confusion-matrix finding that Benign is the hardest class to isolate; PDAC sits at 0.98. The resulting macro-averaged AUC of 0.983 means that, averaged across the three classes, a randomly chosen true-positive sample is ranked above a randomly chosen true-negative sample 98.3% of the time — a strong result given the modest training-set size.

5.3 Feature Importance Analysis

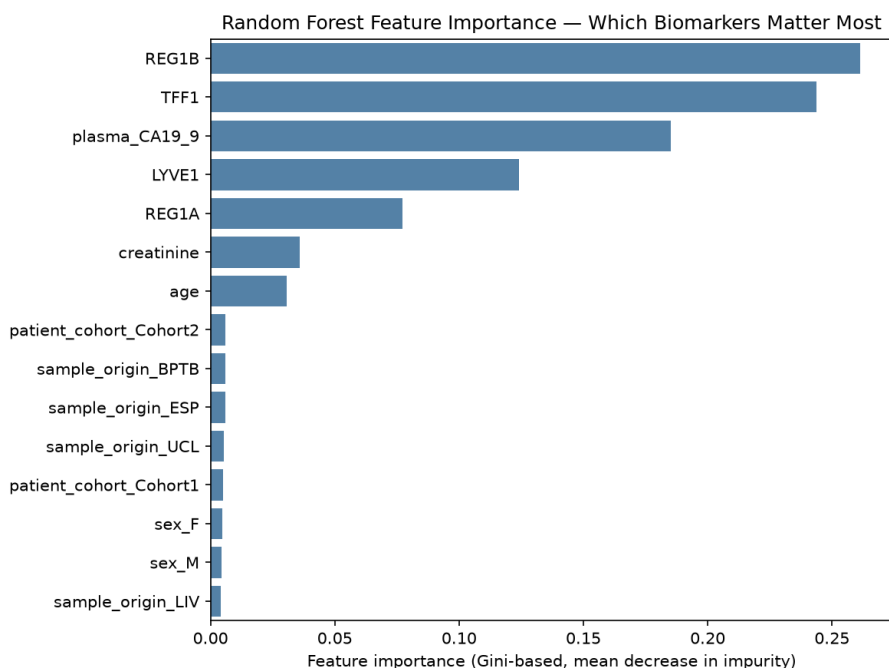


Figure 5. Random Forest Gini-based feature importance, ranking which biomarkers the trained model relies on most.

Feature-importance analysis (Gini-based mean decrease in impurity, per Breiman [2]) ranked REG1B (0.261), TFF1 (0.244), and plasma CA19-9 (0.185) as the three most predictive features, followed by LYVE1 (0.124) and REG1A (0.077). Demographic and batch/site metadata (age, sex, cohort, sample origin) contributed comparatively little to model decisions — collectively under 4% of total importance — which is reassuring, since it indicates the model is relying on biomarker biology rather than on collection-site artifacts or demographic shortcuts.

5.4 Comparison with Prior Literature

Debernardi et al. [1] reported that REG1B and TFF1 were among the strongest individual discriminators in their urinary biomarker panel, and that CA19-9 alone under-performs a combined panel. The independently trained Random Forest in this project recovers the same ranking directly from its Gini-based feature importance, summarized in Table 2:

Table 2. Comparison between this project's Random Forest feature-importance ranking and findings reported by Debernardi et al. [1].

Biomarker	This project's Gini importance	Debernardi et al. finding
REG1B	0.261 (rank 1)	Among top discriminators
TFF1	0.244 (rank 2)	Among top discriminators
Plasma CA19-9	0.185 (rank 3)	Weaker alone than combined panel
LYVE1	0.124 (rank 4)	Contributory, secondary signal

This agreement between an independently trained model and previously published clinical findings gives face validity to the classifier: it demonstrates that the Random Forest is recovering the same underlying biological signal identified in the clinical literature, rather than fitting artifacts specific to this dataset.

6. Limitations

- Sample size ($n = 590$) is modest by machine learning standards, though consistent with typical clinical biomarker study cohorts such as [1].
- Evaluation relies on a single stratified train/test split; repeated cross-validated reporting would tighten confidence intervals on the metrics in Section 5.
- Gini-based feature importance can be biased toward high-cardinality or continuous features relative to categorical ones; permutation importance would provide a useful cross-check.
- No external, site-held-out validation cohort was used; given that samples originate from multiple collection sites, leave-one-cohort-out validation would better estimate cross-site generalization.
- As with the source study [1], CA19-9-based markers are known to be less sensitive in patients with certain blood-group antigen profiles, a subgroup effect not separately modeled here.

7. Conclusion and Future Work

This project demonstrates that a Random Forest classifier, trained on the urinary and plasma biomarker panel established by Debernardi et al. [1] and built according to the ensemble methodology of Breiman [2], can distinguish Control, Benign, and PDAC diagnostic groups with 91.5% accuracy and a macro-averaged ROC-AUC of 0.983. Confusion-matrix and ROC analysis show that the model separates PDAC from Control almost perfectly, with the majority of residual error concentrated on the harder Benign-vs-cancer boundary. Feature-importance analysis identifies REG1B, TFF1, and CA19-9 as the most clinically informative biomarkers, a ranking that agrees with prior clinical findings and thereby lends face validity to the trained model. Future work should prioritize (i) permutation and SHAP-based importance as a cross-check on the Gini-based ranking, (ii) leave-one-cohort-out validation across collection sites, (iii) a focused evaluation on early-stage-only PDAC (Stage I/II) detection, which is the highest-value and most clinically demanding use case, and (iv) probability calibration if model outputs are to inform an actual clinical decision threshold.

References

- [1] S. Debernardi, S. O'Brien, A. G. Algahmdi, et al., "A combination of urinary biomarker panel and PancRISK score for earlier detection of pancreatic cancer: A case-control study," *PLOS Medicine*, vol. 17, no. 6, e1003489, 2020.
- [2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.
- [3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321-357, 2002.
- [4] R. L. Siegel, K. D. Miller, N. S. Wagle, and A. Jemal, "Cancer statistics, 2023," *CA: A Cancer Journal for Clinicians*, vol. 73, no. 1, pp. 17-48, 2023.
- [5] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

Appendix A: Reference Papers

This appendix gives the full bibliographic record, official access link, and a paraphrased summary of the two primary references from Section 2. The publishers' PDFs are not embedded: the environment used to produce this report has no outbound network access to retrieve them, and Breiman (2001) is copyrighted by Springer/Kluwer rather than openly licensed. The summaries below are therefore in our own words, and the DOI links give the authoritative text.

A.1 Debernardi et al. (2020)

Full citation: S. Debernardi, S. O'Brien, A. G. Algahmdi, et al., "A combination of urinary biomarker panel and PancRISK score for earlier detection of pancreatic cancer: A case-control study," PLOS Medicine, vol. 17, no. 6, e1003489, 2020.

DOI / open-access link: <https://doi.org/10.1371/journal.pmed.1003489> — PLOS Medicine publishes under CC BY 4.0, so the publisher's PDF is freely downloadable at this DOI with no subscription or payment required.

Summary (paraphrased, not a direct quotation): the study is a multi-site case-control analysis that measured urinary LYVE1, REG1B, and TFF1 alongside plasma CA19-9 and urinary creatinine in Control, Benign, and PDAC patients, and combined them into a composite risk score that discriminated PDAC more effectively than CA19-9 used alone, including in early-stage disease where CA19-9 is known to underperform.

A.2 Breiman (2001)

Full citation: L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

DOI: <https://doi.org/10.1023/A:1010933404324> (Machine Learning, Springer/Kluwer; available via most university library subscriptions).

Summary (paraphrased, not a direct quotation): the paper formalizes the Random Forest algorithm — an ensemble of decision trees, each trained on a bootstrap-resampled subset of the data with splits restricted to a random subset of features — and shows that this double randomization reduces variance relative to a single decision tree without requiring the careful sequential tuning that boosting methods need. It also introduces out-of-bag error estimation and Gini-based feature importance as natural, low-cost byproducts of training the ensemble, both of which this project relies on directly.